XM · SESIÓN 2 · CLASE 15

# Soluciones de Analítica con Databricks

**Clase 15:** Monitoreo, drift, reentrenamiento y cierre

*Docente: Manuela Larrea Gómez*

# Agenda de la sesión 2 (clase 15)

3 h 30 · el modelo se monitorea, se reentrena y se promueve solo si gana · cierre del curso

| Min | Bloque | Qué pasa |
|---|---|---|
| **10** | Apertura | qa y prod en el mismo commit; la pregunta que falta |
| **30** | Qué se monitorea y con qué | cuatro capas, WAPE y sesgo, drift y PSI, Data quality monitoring, el monitor en el job |
| **35** | Notebook: error y drift | secciones 1–3 del lab 15, con un Niño simulado |
| **10** | Receso | |
| **20** | Políticas de reentrenamiento y promoción | cuatro disparadores · reentrenar no es promover · el sector |
| **40** | TODO + job + reentrenamiento por evento | decidir_reentrenamiento() · deploy a dev · job reentrenamiento |
| **10** | Receso (mientras entrena el candidato) | |
| **25** | Alerta SQL, SLOs y runbook | sección 6 · docs/runbook.md |
| **30** | Cierre del curso | lo que construimos · lo que se llevan · backlog · retro |

# Ya sabemos que corre y que llegó. Falta saber si sigue siendo bueno

Lo que dejó la sesión 1, y la pregunta que ningún control anterior responde

### ¿Qué versión corre?

CLASE 14

qa y prod en el mismo commit (git_sha), desplegados por el pipeline y con una aprobación registrada.

### ¿Llegó a tiempo?

CLASE 12

El chequeo de frescura corre siempre y convierte la falla silenciosa en una falla visible.

### ¿Sigue acertando?

HOY

Nadie lo mide todavía. Un modelo de demanda no falla en rojo: se degrada en silencio cuando el mundo cambia.

Un pipeline verde con datos frescos puede estar publicando un pronóstico malo. El chequeo de la clase 12 mira el calendario; hoy miramos el error y el mundo.

*La pregunta de hoy: ¿cómo se entera el equipo de que el modelo ya no sirve, antes que planeación? Y cuando se entera, ¿quién decide reemplazarlo, con qué evidencia y dónde queda escrito?*

# Qué se monitorea en un modelo en producción

Cuatro capas, en el orden en que avisan: el sistema y los datos primero; el error observado confirma después

### Sistema

*¿corre, a tiempo y a qué costo?*

fallas y reintentos · duración · DBUs

Notificaciones · avisos por duración · system.billing.usage

### Datos

*¿llegó, completo y a tiempo?*

frescura (clase 12) · volumen · cuarentena (clase 5)

Data quality monitoring: anomaly detection

### Entradas del modelo

*¿se parece al entrenamiento?*

drift por feature: PSI · sin escala · no necesita el dato real

Data profiling: _drift_metrics · monitoreo_drift

### Desempeño

*¿el pronóstico acierta?*

WAPE y sesgo · por segmento · llega 7 días después

Data profiling InferenceLog · monitoreo_error

avisa antes ——————————————————————————► más tarde, y más concluyente

### El drift avisa antes que el error

El error necesita el dato real, que llega 7 días después del pronóstico. El drift solo necesita las entradas de hoy: es la alarma temprana, no la sentencia.

### El promedio esconde

Un WAPE global estable puede ocultar un mercado que se degradó (clase 9). Todo indicador de desempeño se mira también por segmento: tipo de mercado, CIIU, operador.

*Fuente: D. Sculley et al., Hidden Technical Debt in Machine Learning Systems, NeurIPS 2015 · E. Breck et al., The ML Test Score, Google, 2017*

# Error observado: cuánto se equivoca y hacia dónde

src/xm_demanda/monitoreo/metricas.py · error_diario() y wape_ventana() sobre v_observado_vs_estimado

```
monitoreo/metricas.py

# por día y alias, solo filas con observado
WAPE  = Σ |estimado − observado| / Σ observado
sesgo = Σ (estimado − observado)  / Σ observado

# ventana de 7 días: ponderada por energía
wape_7d = Σ wape·energía / Σ energía
```

| Si ves… | Lectura |
|---|---|
| **WAPE sube, sesgo ≈ 0** | más ruido: el modelo duda |
| **WAPE sube, sesgo > 0 sostenido** | sobreestima: la demanda bajó |
| **WAPE sube, sesgo < 0 sostenido** | subestima: cambio estructural al alza |
| **WAPE estable, un segmento peor** | el promedio esconde: ver por mercado |

### ¿Por qué ponderar por energía?

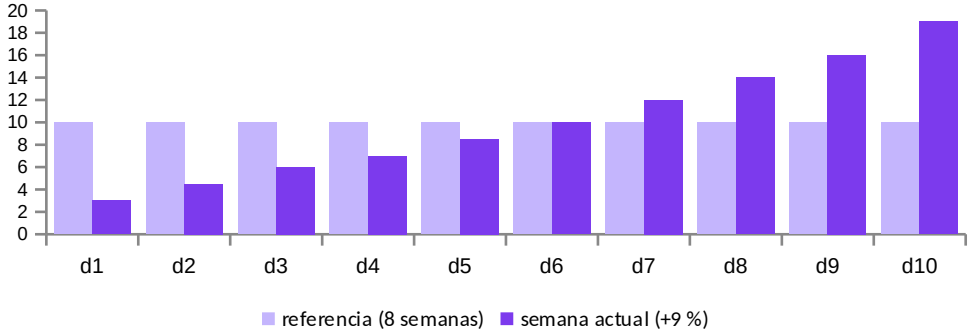
Un festivo de baja demanda o una serie de 30 kWh no deben pesar lo mismo que un martes en Bogotá. Promediar porcentajes diarios es el error que Genie no cometió en la clase 12 gracias a la vista curada.

### El sesgo es la primera huella

Un sesgo con el mismo signo durante una semana casi nunca es ruido: es la demanda moviéndose (un Niño, una carga industrial nueva). El WAPE dice cuánto; el sesgo dice hacia dónde.

# Drift con PSI: ¿el mundo de hoy se parece al del entrenamiento?

Semana actual vs. 8 semanas anteriores, por feature · psi() y drift_features() en monitoreo/metricas.py




```
PSI = Σ (actual − ref) · ln(actual / ref)
```

| PSI | Lectura | En la política |
|---|---|---|
| < 0,10 | estable | nada |
| 0,10 – 0,25 | moderado | mirar error y segmento |
| ≥ 0,25 | alto | dispara reentrenamiento |

### Sobre features sin escala

demanda / media de 28 días de su serie. En kWh, un cambio en la mezcla de 355 series (30 kWh a 35 GWh) parecería drift sin serlo.

### Por qué PSI

Nació en riesgo de crédito y se volvió estándar por algo práctico: se lee igual para cualquier variable. Databricks también calcula KS, Wasserstein y Jensen-Shannon.

### No todo drift daña

Diciembre siempre se ve distinto a noviembre. Por eso el drift dispara un reentrenamiento, no una promoción: el candidato todavía tiene que ganar.

# Data quality monitoring en Databricks, y nuestro monitor

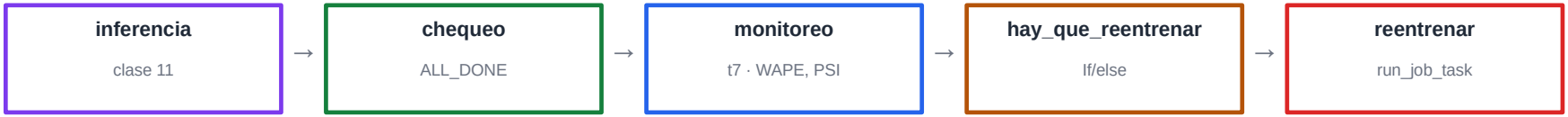
La oferta administrada y el monitor como código no compiten: se complementan

| | Anomaly detection | Data profiling (antes Lakehouse Monitoring) | Monitor propio (hoy) |
|---|---|---|---|
| **Qué mide** | frescura y completitud de cada tabla | perfil y drift por columna; InferenceLog: error del modelo | WAPE, sesgo, PSI y la decisión de reentrenar |
| **Cómo se activa** | un clic sobre el esquema (Catalog Explorer) | API w.data_quality o UI, por tabla | tarea del job, en src/ con tests |
| **Salida** | estado en Catalog Explorer; system.data_quality_monitoring | _profile_metrics, _drift_metrics y un tablero | monitoreo_error, _drift, _decisiones |
| **Decide algo** | alerta | alerta | sí: task value reentrenar → run_job_task |
| **Free Edition** | puede no estar disponible | puede no estar disponible | sí |

En XM: anomaly detection sobre los esquemas de producción y data profiling InferenceLog sobre pronostico_demanda, más la política como código que decide.

# El monitor dentro del job: medir, decidir y actuar

resources/job_diario.yml · tres tareas nuevas después del chequeo




resources/job_diario.yml

```yaml
- task_key: monitoreo
  depends_on: [ chequeo ]      # solo lo publicable
- task_key: hay_que_reentrenar
  condition_task:
    left: {{tasks.monitoreo.values.reentrenar}}
    op: EQUAL_TO   right: si
- task_key: reentrenar
  depends_on: [ hay_que_reentrenar: true ]
  run_job_task:
    job_id: ${resources.jobs.reentrenamiento.id}
```

**job reentrenamiento**

entrenar (t8) → evaluar_promover (t9)
su propio historial, alertas y disparo manual

**¿Por qué otro job y no una tarea más?**

El reentrenamiento tarda, cuesta y se lanza también por evento (una persona). Separado, tiene su propio historial y sus alertas, y el diario no espera una hora si algo se cuelga.

# Cuatro tablas: el monitoreo deja evidencia, no solo alertas

En el esquema de cada ambiente. Lo que no queda escrito no se puede auditar ni enseñar

| Tabla | Qué guarda | Cómo se escribe | Quién la lee |
|---|---|---|---|
| **monitoreo_error** | WAPE y sesgo por día y alias | se recalcula completa (idempotente) | tablero, Genie, alerta SQL |
| **monitoreo_drift** | PSI por feature y su nivel | una foto por corrida (append) | quien investiga un cambio |
| **monitoreo_decisiones** | reentrenar sí/no, con motivos, WAPE, PSI y días | solo se agrega | auditoría, runbook |
| **decisiones_modelo** | champion vs. candidato: días, WAPE de cada uno, promovido, motivo | solo se agrega | auditoría, líder técnico |

Seis meses después, alguien pregunta por qué cambió el pronóstico en octubre. La respuesta no es un recuerdo: es una fila con fecha, evidencia y motivo, y el alias que se movió.

# Cuatro disparadores de reentrenamiento: basta uno

decidir_reentrenamiento() · umbrales de negocio en conf/base.yml, no en el código

| Disparador | Regla | A favor | En contra |
|---|---|---|---|
| **Degradación** | WAPE 7 d > validación × 1,20 | mide lo que importa | llega tarde |
| **Drift** | PSI máximo ≥ 0,25 | llega temprano | no todo drift daña |
| **Calendario** | ≥ 28 días de datos nuevos | predecible | reentrena sin necesidad |
| **Evento** | una persona lo lanza | sabe lo que el dato no | depende de que avisen |

conf/base.yml

```yaml
monitoreo:
  ventana_reciente_dias: 7
  tolerancia_degradacion: 0.20
  umbral_critico: 2.0
  psi_alto: 0.25
  dias_max_sin_reentrenar: 28
promocion:
  mejora_minima: 0.02
  dias_minimos: 14
```

TODO del lab: tres disparadores independientes, un nan no dispara nada, y cada uno deja un motivo legible. El cuarto, el evento, no es código: es el job lanzado a mano con su motivo.

# Reentrenar no es promover

Entrenar produce un @candidato. Reemplazar al champion lo decide otra regla, con evidencia

**Degradación** WAPE 7 d > validación × 1,20

**Drift** PSI máximo ≥ 0,25

**Calendario** ≥ 28 días de datos nuevos

**Evento** una persona, con un motivo

### `decidir_reentrenamiento()`

Basta un disparador; un nan no dispara. → job reentrenamiento: entrena un @candidato con los datos del ambiente. No toca @champion.

### `decidir_promocion()`

Mismos días, que ninguno vio. Promueve con ≥ 14 días y ≥ 2 % de mejora. Si promueve: @anterior. Siempre: una fila en decisiones_modelo.

### Comparar sin fuga

Solo en días que ninguno de los dos vio al entrenar: después del datos_hasta más reciente. Por eso cada versión guarda su linaje en tags.

### Evidencia y margen

Al menos 14 días comparables y al menos 2 % de mejora relativa. Un mes bueno no basta (lo vimos en la clase 9 con el challenger de julio).

### Volver atrás es mover un alias

Si promueve, el champion anterior queda como @anterior. Sin redesplegar nada: la siguiente corrida ya usa la versión vieja.

# Qué degrada un modelo de demanda en Colombia

Cambios estructurales: el modelo aprendió un mundo que ya no existe

**+2,3 %**
demanda del SIN en 2024: 82.084,9 GWh

**9 + 4**
meses de El Niño en 2023 y en 2024

**+8,76 %**
demanda de mayo de 2026 vs. 2025: récord histórico

**69,43 %**
embalses al 4 de junio de 2026, bajo la senda

### Clima e hidrología

El Niño cambia temperatura, consumo y precios. En 2024 el regulado llegó a +12,2 % en marzo mientras el no regulado cerró el año en −1,83 %: el mismo choque, efectos opuestos por segmento.

### Autogeneración solar

La demanda que ve el SIN baja de día donde crece la autogeneración, sin que el consumo real baje. El sesgo aparece por mercado y por hora antes que en el total.

### Cargas y reglas nuevas

Una planta industrial, un centro de datos, un cambio regulatorio en la clasificación de usuarios: escalones que ningún lag anticipa. Son el disparador por evento.

Ninguno de estos cambios rompe el job. Todos aparecen primero como sesgo sostenido o como drift en un segmento.

*Fuente: XM, vía Valora Analitik (2025): demanda 2024 · XM, vía Forbes Colombia (11-jun-2026): crecimiento 2026, embalses y riesgo de El Niño*

# SLOs y una alerta SQL: promesas medibles

Sin un objetivo con tolerancia, "el pronóstico está bien" es una opinión

| SLO | Objetivo | Se mide en |
|---|---|---|
| **Frescura** | publicable antes de las 8:00, 95 % de días hábiles | chequeo |
| **Calidad** | WAPE 7 días del champion ≤ 5,8 % | monitoreo_error |
| **Estabilidad** | PSI < 0,25 en todas las features | monitoreo_drift |
| **Costo** | job diario dentro de su presupuesto de DBUs | system.billing.usage (Azure) |

SQL Editor

```
-- Alerts → Create alert
SELECT sum(wape * energia_gwh)
     / sum(energia_gwh) AS wape_7d
FROM …monitoreo_error
WHERE alias = 'champion'
  AND fecha_pronostico > max − 7 días
-- condición: wape_7d > 0.058
```

**0,058 no es un número mágico**

Es el WAPE de validación (4,8 %) con la tolerancia de 20 %: el mismo umbral que la política. Alerta y política dicen lo mismo.

*Fuente: Google, Site Reliability Engineering, cap. 4: Service Level Objectives*

# Runbook: qué hacer a las 6:15 de un lunes festivo

docs/runbook.md · cada alerta con dueño, primeros 30 minutos y a quién escalar

| Alerta | Señal | Primer paso |
|---|---|---|
| **A1 No publicable** | chequeo falla | ¿llegó el archivo? → Repair run |
| **A2 Cuarentena alta** | calidad > 1 % | ¿fuente o republicación? |
| **A3 WAPE crítico** | monitoreo > 2× validación | ¿un segmento o todos? → @anterior |
| **A4 Drift o degradación** | reentrenar se dispara | leer motivos y decisión |
| **A5 Reentrenamiento falla** | job reentrenamiento | el champion sigue sirviendo |
| **A6 Promoción detenida** | deploy-qa/prod en rojo | nunca desplegar prod a mano |

| Volver atrás | Cómo |
|---|---|
| **Modelo** | @champion ← @anterior |
| **Código** | re-run del tag anterior |
| **Datos** | RESTORE … VERSION AS OF |

### En el lab

Cada <dueño> se reemplaza por un rol de tu equipo. Si no hay a quién ponerle una alerta, esa alerta no debería existir.

# Lab 15: el modelo se monitorea, se reentrena y se promueve solo si gana

clases/c15-monitoreo/notebooks/lab15_monitoreo_reentrenamiento · widget usuario · rama feature/c15-monitoreo

1

### Error observado y segmentos

WAPE de 7 y 28 días, sesgo, WAPE por tipo_mercado

2

### Drift y un Niño simulado

PSI por feature; +9 % al regulado en una copia

3

### TODO: decidir_reentrenamiento()

src/xm_demanda/monitoreo/politica.py; test_monitoreo.py en verde

4

### Deploy a dev y reentrenamiento por evento

diario con monitoreo; job reentrenamiento con motivo; decisiones_modelo

5

### Alerta SQL y runbook

wape_7d > 0.058; cada <dueño> con un rol

### Criterios de aceptación

- test_monitoreo.py sin skip
- Diario con monitoreo → reentrenar; job reentrenamiento
- Cuatro tablas de monitoreo con filas
- @candidato con datos_hasta y motivo
- Alerta creada; runbook sin <dueño>
- Dos preguntas; verificar() en OK

### Después: tag v1.1.0

El pipeline lleva el monitor a qa y, con tu aprobación, a prod.

# Lo que construimos en quince clases

Una pregunta de negocio, de un Excel a un pronóstico que se opera solo

### H1 · Pregunta
clases 1–2

Lakehouse, Medallion, ADR-001

### H2 · Fuentes
clases 3–4

Unity Catalog, Auto Loader, republicaciones

### H3 · Calidad
clases 5–6

reglas, cuarentena, MERGE, plata

### H4 · Oro
clases 7–8

features con contrato, src/, tests, CI

### H5 · Modelo
clases 9–11

MLflow, registro, batch, endpoint, app, Genie

### H6 · Operación
clases 12–15

job, bundle, CI/CD, monitor y reentrenamiento

**15**
clases y un solo producto que creció en cada una

**7 + 3**
tareas del job diario, más monitoreo y reentrenamiento

**3**
ambientes desplegados por el mismo bundle

**6**
hitos cerrados, cada uno con su verificar()

El primer día dijimos que el algoritmo importaba poco. Por eso no hablamos de hiperparámetros: hablamos de qué pasa el día que el modelo se equivoca y nadie está mirando.

# Backlog post-curso y próximos pasos

docs/backlog.md · lo que quedó por fuera, ordenado por lo que más aporta en Azure Databricks

| Ítem | Prioridad |
|---|---|
| Federación OIDC o service connection de Entra ID, sin tokens | Alta |
| Service principal por ambiente; grupos dueños de los jobs | Alta |
| Workspaces y catálogos por ambiente con binding (Terraform) | Alta |
| Reentrenar en cada ambiente (deploy code) | Alta |
| Data quality monitoring en producción | Media |
| Features exógenas: temperatura, hidrología, ONI | Media |
| Tablero AI/BI de operación y Genie encima | Media |

1

**Semana 1**

un proyecto real en src/ con tests y CI

2

**Mes 1**

bundle con dev y qa; el job como código

3

**Trimestre**

prod por pipeline con aprobación y monitor

*Fuente: Databricks, The Big Book of MLOps · MLOps Stacks · Google Cloud, MLOps: continuous delivery and automation pipelines in ML*




iData
Global | Data & AI

# Gracias

Fin del curso · Soluciones de Analítica con Databricks

DOCENTE

**Manuela Larrea Gómez**